Informe de seguridad · CaféDirecto

Auditoría con OWASP Mobile Top 10 (2024), correcciones en código y verificación antes y después.

Materia: Programación Móvil

Profesor: Alex Vacca

Grupo: [Nombres de los integrantes]

Entrega: 16 de noviembre de 2026

Proyecto: CaféDirecto 3.2 · app Flutter (Android) + API Node.js/PostgreSQL

Repositorio: github.com/gustavomutto/CafeDirecto · rama `Robert`

Resumen

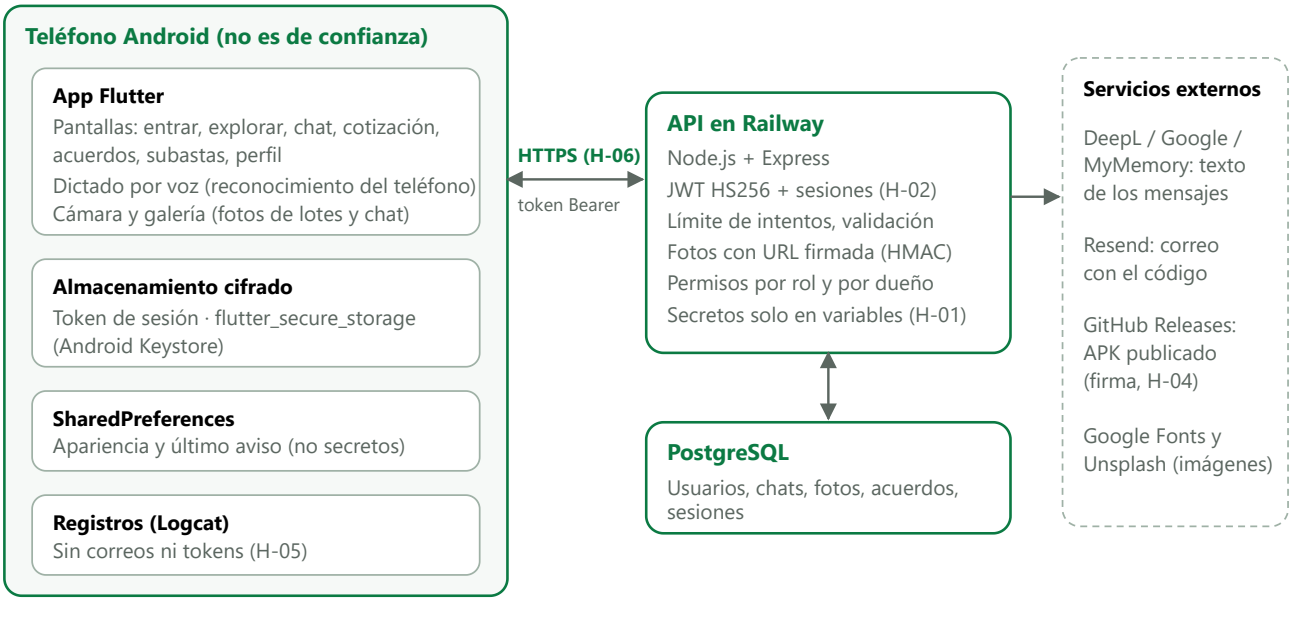
Se auditaron la app Flutter y su API. De los diez riesgos del Top 10, **3 se cumplen** y **7 tenían algo por corregir**. Se registraron **8 hallazgos**: **7 se corrigieron**, cada uno en su propio commit con su ID, y se verificaron repitiendo la misma prueba. El hallazgo que queda abierto es de severidad baja y tiene un plan.

8	7	5 de 5	159 + 39
hallazgos registrados	corregidos y verificados (mínimo pedido: 5)	mejoras obligatorias cumplidas	pruebas automáticas en verde (API + app)

Los dos hallazgos más graves estaban en el servidor. Uno era una **clave de firma de sesiones publicada en el repositorio** (H-01): con ella cualquiera entraba como cualquier usuario. El otro, **sesiones que no se podían cerrar** (H-02): un token robado seguía sirviendo 90 días después de cerrar sesión.

Fase 1 · Modelo de amenazas

Diagrama de la app y flujo de datos



Regla base del data sheet: el teléfono y la app instalada no son de confianza. Por eso cada decisión (quién ve un chat, quién marca un acuerdo como entregado, quién descarga una foto) se valida en el servidor.

Datos sensibles que maneja la app

Dato	Dónde está	Por qué es sensible
Contraseña	Solo viaja al servidor; se guarda con bcrypt	Da acceso a la cuenta (y suele repetirse en otros servicios)
Token de sesión (JWT)	Keystore del teléfono · cabecera de cada petición	Quien lo tenga actúa como el usuario
Código de recuperación	Correo del usuario · hash en la base	Permite cambiar la contraseña
Correo, teléfono y nombre	Base de datos · perfil	Datos personales (Ley 1581 de 2012)
Ubicación de la finca	Departamento y municipio en el perfil	Ubica a una persona y su producción
Chats, fotos y precios negociados	Base de datos · pantalla del teléfono	Información comercial privada entre caficultor y comprador
Voz del usuario	Se procesa en el teléfono; solo llega el texto	Dato biométrico si se enviara

Amenazas propias de CaféDirecto

#	Amenaza	Quién podría aprovecharla	Por dónde · qué ganaría	Control
A1	Falsificar sesiones y entrar como otro usuario	Cualquiera que lea el repositorio público	Firmar un JWT con la clave de desarrollo publicada; ver chats, precios y cerrar acuerdos a nombre de otro	H-01
A2	Robo o copia del token de sesión	Otra persona con acceso al teléfono, una app maliciosa o alguien que vio un registro	El token sigue sirviendo aunque el dueño cierre sesión; suplantación durante semanas	H-02, H-05
A3	APK modificado o falso	Un intermediario de compra que quiera engañar a caficultores	Reempaquetar la app (lógica legible, firma de depuración) y distribuirla por WhatsApp como "versión nueva" para robar credenciales	H-03, H-04
A4	Interceptar la red	Quien controle una wifi pública rural o de un evento cafetero	Leer contraseñas y mensajes si la app usara http	H-06
A5	Abusar de la API	Bots	Adivinar contraseñas, crear cuentas masivas, gastar la traducción	Ya controlado: límites de intentos

Fase 2 · Auditoría con OWASP Mobile Top 10 (2024)

Técnicas usadas (se pedían al menos dos; se usaron cinco)

Técnica	Cómo se hizo
1. Búsqueda de secretos en el código y en el historial de Git	Script propio (`herramientas/secret-scan.py`) que revisa cada línea agregada en los 33 commits de todas las ramas con patrones de claves (AWS, GitHub, Google, Resend, JWT, URL con contraseña, asignaciones de secretos). Se reemplazó a gitleaks porque no se usó Docker.
2. Análisis del APK de lanzamiento	`flutter build apk --release` y luego `aapt2 dump xmltree` (manifest), `apksigner verify --print-certs` (firma), búsqueda de nombres dentro de `libapp.so` (ofuscación) y `mapping.txt` (R8).
3. Revisión de dependencias	`npm audit --omit=dev` en el servidor y `flutter pub outdated` en la app.
4. Revisión de `AndroidManifest.xml` y permisos	Permisos pedidos, componentes exportados, copias de seguridad, tráfico en claro.
5. Pruebas dinámicas contra la API local	Script (`herramientas/security-check.mjs`) que intenta cada ataque contra el servidor con PostgreSQL local y dice si sigue abierto; más las 150 pruebas existentes de la API.

> No se usó Logcat ni mitmproxy porque el equipo no tiene emulador Android ni teléfono conectado. La versión de depuración ya queda preparada para mitmproxy (H-06 acepta certificados del usuario solo en depuración).

Revisión de los diez riesgos

ID	Riesgo	Estado	Justificación
M1	Improper Credential Usage	**No cumple**	La clave JWT de respaldo estaba escrita en el código y publicada; `start-local.cmd` traía claves (H-01). No hay llaves de servicios de terceros en el código ni en el historial.
M2	Inadequate Supply Chain Security	**No cumple (parcial)**	Servidor: `npm audit` = 0 vulnerabilidades. App: 12 dependencias transitivas desactualizadas (H-07).
M3	Insecure Authentication/Authorization	**No cumple**	El servidor valida sesión y permisos en cada petición, pero cerrar sesión no invalidaba el token y este duraba 90 días (H-02).
M4	Insufficient Input/Output Validation	**Cumple**	El servidor valida tipo y rango de cada campo (`inRange`, reglas `CHECK`), revisa el tipo real de las fotos y que cada foto sea del usuario. La app valida correo y contraseña. No hay deep links ni WebView.
M5	Insecure Communication	**No cumple (parcial)**	La API usa HTTPS y Android 9+ bloquea http por defecto, pero no había configuración explícita y la app aceptaba un servidor http en la versión publicada (H-06).

M6	Inadequate Privacy Controls	**No cumple (parcial)**	Hay política de privacidad, eliminación de cuenta y permisos justificados (micrófono para dictado). Pero la app escribía el correo del usuario en los registros (H-05).
M7	Insufficient Binary Protections	**No cumple**	R8 activo para Java/Kotlin, pero el código Dart no estaba ofuscado: nombres como `SessionLocalDataSource` o `saveToken` se leían en el binario (H-03).
M8	Security Misconfiguration	**No cumple**	Sin modo debug en release, copias de seguridad apagadas y solo la actividad principal exportada. Pero el APK público iba firmado con la clave de depuración (H-04).
M9	Insecure Data Storage	**Cumple**	El token va en Android Keystore con `flutter_secure_storage` 11 (no usa la librería obsoleta EncryptedSharedPreferences); `allowBackup=false`. Queda una mejora baja: proteger la pantalla de capturas (H-08).
M10	Insufficient Cryptography	**Cumple**	bcrypt para contraseñas y códigos, HMAC-SHA256 para firmar fotos, JWT solo HS256, números aleatorios con `crypto`. No se usa MD5 ni SHA-1.

Hallazgos

ID	Riesgo	Descripción	Evidencia	Severidad	Corrección propuesta
H-01	M1	Sin `JWT_SECRET`, fuera de Railway el servidor usaba una clave fija publicada en GitHub	Token firmado con esa clave → **HTTP 200**, entra como el usuario (`server-antes.txt`)	**Alta**	Sin clave fija: no arrancar sin `JWT_SECRET`; en desarrollo, clave aleatoria; `.env.local` fuera de git
H-02	M3	Cerrar sesión no invalidaba el token en el servidor; vigencia de 90 días	Después de "cerrar sesión" el mismo token → **HTTP 200**; `/auth/logout` → 404	**Alta**	Sesiones en la base, token con id de sesión, `POST /auth/logout`, 30 días
H-03	M7	Código Dart sin ofuscar en el APK de lanzamiento	Nombres legibles en `libapp.so`: `ChatRoomController` ×2, `SessionLocalDataSource`, `saveToken`… (`obfuscacion-antes.txt`)	**Media**	`--obfuscate --split-debug-info` en CI y en la guía de publicación
H-04	M8	APK de lanzamiento firmado con la clave de depuración y publicado en Releases	`apksigner`: **CN=Android Debug** (`firma-antes.txt`)	**Media**	Firma desde secretos de GitHub; un Release sin clave falla; aviso en Gradle
H-05	M6	La app escribía el correo del usuario en los registros	4 líneas de código; prueba: "Entró lena@nordlicht.de (buyer)" (`logs-antes.txt`, `h05-prueba-antes.txt`)	**Media**	Registrar el id del usuario, nunca el correo
H-06	M5	HTTPS no forzado por configuración; un release con `API_URL=http://` funcionaría	Manifest del APK sin `networkSecurityConfig`; `env.dart` acepta cualquier esquema (`red-antes.txt`)	**Baja**	Network Security Config sin tráfico en claro; la app no arranca con http en release
H-07	M2	12 dependencias transitivas de la app desactualizadas	`flutter pub outdated` (`dependencias-app-antes.txt`)	**Baja**	`flutter pub upgrade` dentro de las versiones permitidas

H-08	M9	Sin protección de pantalla: chats y precios aparecen en capturas y en "apps recientes"	`MainActivity.kt` sin `FLAG_SECURE`	**Baja**	`FLAG_SECURE` en las pantallas de chat, cotización y contraseña. **Abierto**: requiere un teléfono o emulador para verificarlo

Fase 3 · Correcciones

Cada corrección es un commit en la rama `Robert` cuyo mensaje empieza con el ID del hallazgo.

ID	Commit	Qué se cambió
H-01	`e4df2cb`	`lib/secrets.js` sin clave fija; sin `JWT_SECRET` el proceso termina, salvo con `NODE_ENV=development/test` (clave aleatoria). `start-local.cmd` crea `.env.local` con clave aleatoria; `.env.example`; pruebas con `NODE_ENV=test`.
H-02	`a642936`	Tabla `sessions`; `issueToken()` crea la sesión y la incluye (`sid`) en el token; `requireAuth` exige sesión vigente; `POST /auth/logout`; reset de contraseña revoca todas; vigencia 30 días. La app llama a `/auth/logout` al salir. 9 pruebas nuevas.
H-03	`35a21a4`	CI compila con `--obfuscate --split-debug-info=build/symbols` y guarda los símbolos como artefacto aparte; misma instrucción en la guía de Google Play.
H-04	`e1da287`	`build.gradle.kts` avisa al firmar un release con la clave de depuración y falla si se exige firma. CI firma con los secretos `ANDROID_KEYSTORE_*` y no publica un Release sin ellos.
H-05	`6b2804a`	Los 4 registros con correo pasan a usar el id del usuario; prueba que falla si aparece un correo en los registros.
H-06	`03b740d`	`network_security_config.xml` (sin http, solo certificados del sistema); versión de depuración con http solo al equipo local; `insecureApiUrlReason()` y bloqueo al arrancar; 3 pruebas.
H-07	`403da4e`	`flutter pub upgrade`: 12 dependencias al día sin cambiar `pubspec.yaml`.

Mejoras obligatorias

Mejora	Estado	Cómo se cumple
Ningún secreto en el código ni en el repositorio; si se publicó, rotarlo	**Cumplida**	H-01. La clave publicada era solo de desarrollo y nunca se usó en Railway, que exige su propia `JWT_SECRET`; ya no firma tokens válidos (prueba en `v5.test.mjs`). No hay otras claves en los 33 commits.
Tokens y datos sensibles con Keystore o Keychain	**Cumplida**	Ya estaba: `flutter_secure_storage` 11 (Android Keystore) con migración desde SharedPreferences.
Todo el tráfico por HTTPS, sin tráfico en claro en release	**Cumplida**	H-06.
Validación de entradas en la app y de permisos en el servidor	**Cumplida**	Revisada en M4: validación por campo y rango en la API, permisos por rol y dueño en cada ruta (pruebas existentes de permisos), validadores en la app.
Release sin modo debug, sin logs sensibles y con R8	**Cumplida**	Sin `debuggable` en el APK; R8 activo (`mapping.txt`); sin correos en registros (H-05); ofuscación de Dart (H-03).

Fase 4 · Verificación antes y después

Para cada hallazgo se repitió exactamente la misma prueba. Un hallazgo se da por cerrado solo si la prueba ya no lo reproduce.

ID	Prueba repetida	Antes	Después	Resultado
H-01	Token firmado con la clave publicada; servidor sin `JWT_SECRET`	**HTTP 200 · entra como el usuario**	**El servidor no arranca (código 1); en modo prueba el token → HTTP 401**	**Cerrado**
H-02	Iniciar sesión, cerrar sesión y reusar el token	**logout → 404; token → HTTP 200; 90 días**	**logout → 200; token → HTTP 401; 30 días; la otra sesión sigue abierta**	**Cerrado**
H-03	Buscar nombres del código en `libapp.so`	**Visibles: ChatRoomController ×2, SessionLocalDataSource, AuthRepositoryImpl, ApiClient ×2, saveToken, readToken**	**0 de esos nombres. Solo queda el texto literal `cafedirecto-token` (nombre de la clave de almacenamiento, no es un secreto)**	**Cerrado**
H-04	`apksigner verify --print-certs`	**CN=Android Debug**	**CN=CafeDirecto Clave de PRUEBA (clave propia, creada para la verificación) · release exigido sin clave → falla**	**Cerrado**
H-05	Prueba: entrar y revisar los registros	**Falla: "Entró lena@nordlicht.de (buyer)"**	**Pasa: ningún registro con correo**	**Cerrado**
H-06	aapt2 sobre el manifest del APK + pruebas de URL	**0 referencias a `networkSecurityConfig`**	**`networkSecurityConfig` en el manifest; `cleartextTrafficPermitted=false` y solo certificados del sistema; release con http no arranca**	**Cerrado**
H-07	`flutter pub outdated`	**12 actualizables bloqueadas**	**0 actualizables (solo queda google_fonts, versión mayor)**	**Cerrado**

Salida de las pruebas (extracto)

```
$ node herramientas/security-check.mjs server       # ANTES
H-01 VULNERABLE  token falsificado con la clave pública del repo → HTTP 200
H-02 VULNERABLE  cerrar sesión → HTTP 404; el mismo token después → HTTP 200; vigencia del token: 90 días

$ node herramientas/security-check.mjs server       # DESPUÉS
H-01 CORREGIDO   token falsificado con la clave pública del repo → HTTP 401
H-02 CORREGIDO   cerrar sesión → HTTP 200; el mismo token después → HTTP 401; vigencia del token: 30 días

$ npm test   (API contra PostgreSQL)                 # DESPUÉS
flow 41 ok · v13 27 ok · v2 43 ok · v3 24 ok · v4 15 ok · v5 9 ok  → 159 pruebas, 0 fallas

$ flutter analyze && flutter test                    # DESPUÉS
No issues found! · 39 pruebas, All tests passed!
```

La evidencia completa está en `docs/seguridad/evidencia/` del repositorio.

Recomendaciones y mejoras pendientes

#	Recomendación	Por qué
1	Cerrar H-08: `FLAG_SECURE` en chat, cotización y contraseña	Evita que los precios y la negociación queden en capturas o en la vista de apps recientes. Hay que verificarlo en un teléfono.
2	Cargar los secretos de firma en GitHub y crear la clave de lanzamiento	Desde H-04, un Release sin ellos ya no se publica.
3	Certificate pinning con plan de rotación (mejora opcional)	Protege contra certificados emitidos a terceros; requiere un plan para cuando Railway renueve el certificado.
4	Play Integrity en las acciones críticas (mejora opcional)	Detecta APK modificados como el de la amenaza A3.
5	Actualizar `google_fonts` de 6 a 9	Cambio de versión mayor; requiere revisar la API de fuentes.

> **Al desplegar en Railway:** después de H-02, los tokens anteriores (sin id de sesión) dejan de servir y cada usuario debe iniciar sesión una vez de nuevo.

Demostración de 10 minutos

Minuto	Quién	Qué se muestra
0:00–1:30	[Integrante 1]	La app y su diagrama. Regla base: el teléfono no es de confianza. Datos sensibles y las tres amenazas principales.
1:30–3:00	[Integrante 2]	Tabla del Top 10: qué cumple y qué no, y las cinco técnicas usadas.
3:00–5:30	[Integrante 3]	**Hallazgo 1 en vivo (H-01).** `git checkout e731dd0` (último commit antes de las correcciones), levantar la API y correr `security-check.mjs`: "VULNERABLE, HTTP 200". Volver a `Robert`: el servidor no arranca sin clave y el mismo ataque da 401. Mostrar el commit `e4df2cb`.
5:30–8:00	[Integrante 4]	**Hallazgo 2 en vivo (H-02).** Mismo script: antes el token sirve después de cerrar sesión; después responde 401. Mostrar `v5.test.mjs` en verde y el commit `a642936`.
8:00–9:30	[Integrante 1]	APK antes y después: `apksigner` (firma) y nombres en `libapp.so` (ofuscación).
9:30–10:00	Todos	Pendientes (H-08, pinning, Play Integrity) y preguntas.

Conceptos del data sheet en CaféDirecto

Concepto	Ejemplo en la app
Superficie de ataque	APK público en GitHub Releases, token en el teléfono, la API en Internet, las fotos por URL, los servicios de traducción.
Modelo de amenazas	Las amenazas A1–A5 de la Fase 1, hechas antes de elegir controles.
Mínimo privilegio	Solo se piden internet, micrófono (dictado) y notificaciones. En la API, solo el caficultor despacha y solo el comprador marca la entrega.
Defensa en profundidad	HTTPS + token cifrado + sesión revocable + permisos en el servidor + límite de intentos: si falla una capa, las otras siguen.
Datos en reposo y en tránsito	En reposo: token en Keystore y contraseñas con bcrypt. En tránsito: HTTPS obligatorio (H-06).
Sandbox	Otras apps no leen el almacenamiento de CaféDirecto; con root sí, por eso el token además va cifrado y la sesión se puede revocar.

Pruebas hechas solo sobre el proyecto del grupo y su servidor local, con cuentas inventadas (`@example.test`). Ningún secreto real ni dato personal aparece en este informe ni en la evidencia. Referencias: OWASP Mobile Top 10 (2024), OWASP MASVS y MASTG.